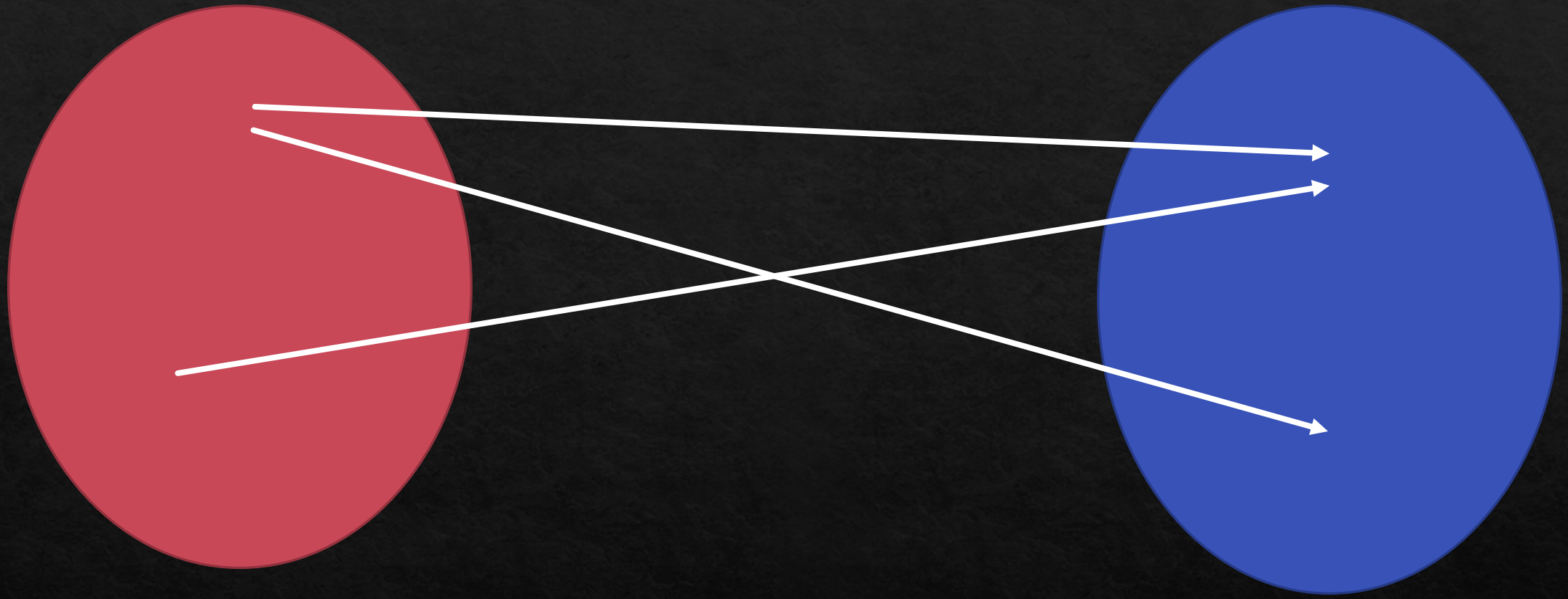
Хеш-таблицы

хеширование, разрешение коллизий, метод цепочек и метод «открытого ключа»

Хеш-функция

Хеш-функция



Чаще всего хеш-функция отображает более мощные множества в менее мощные и является инъекцией. Желательно, чтобы хеш-функция отображала «близкие» объекты из множества определения в «далёкие» объекты из множества значений. Желательно, чтобы прямое отображение было простым, а обратное – сложным.

Хеш-функция может использоваться:

1. для контроля целостности данных;
2. подтверждения верных данных без их явного сравнения;
3. подтверждения проведённой работы.



Задана строка haystack. Требуется узнать есть ли внутри неё подстрока, совпадающая со строкой needle.
Асимптотическая сложность наивного алгоритма O(L(h) * L(n)), где L – длина строки.

Идея алгоритма Рабина-Карпа состоит в том, чтобы сравнивать не сами строки, а их хеши, вычисленные специальным способом.



$$H = s_0 b^{m-1} + s_1 b^{m-2} + \cdots + s_{m-1} b^0$$

$$H_{m+1} = H_m b + s_m$$

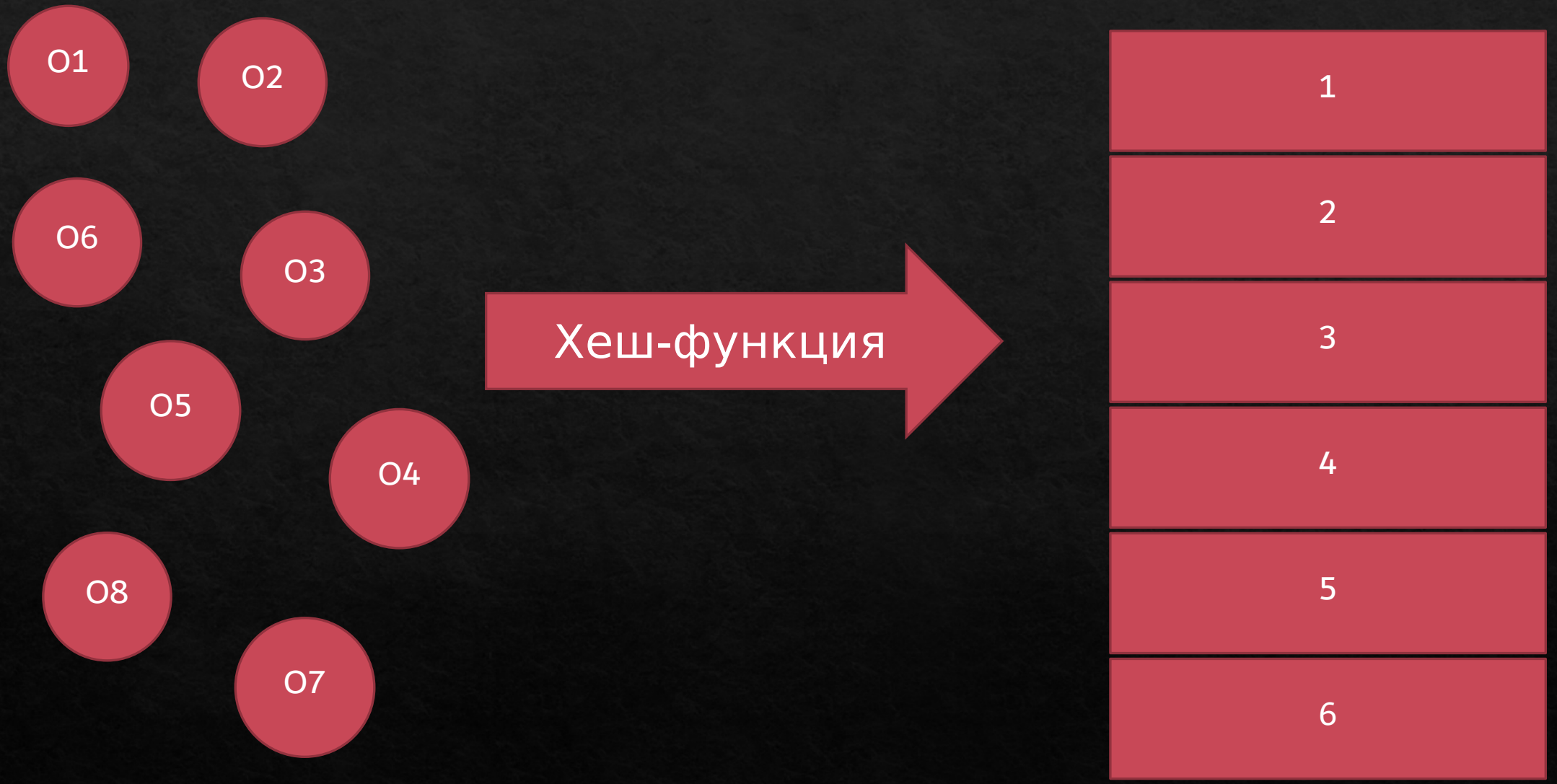
$$H_{n+1} = \left(H_n - a b^l\right) b + x$$

Избежать работы с колоссальными целочисленными значениями можно перейдя в арифметику по модулю простого основания Q.

В случае совпадения хешей можно проверить совпадение строк прямым сравнением.

Хеш-таблица

Хеш-таблица

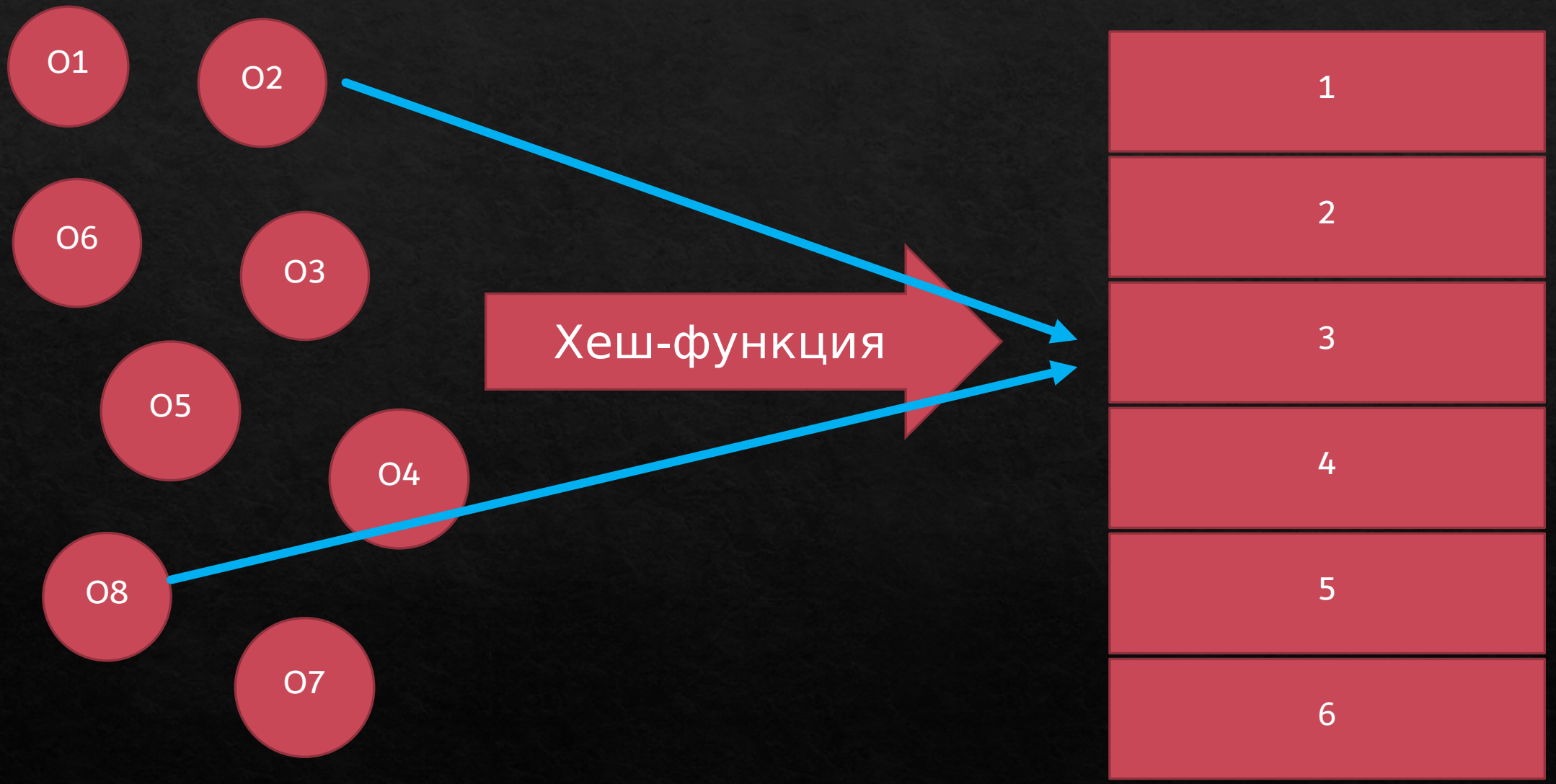


Хеш-таблица

Для добавления элемента в хеш-таблицу требуется определить хеш-функцию и возможность сравнения объектов на точное совпадение на случай «коллизии».

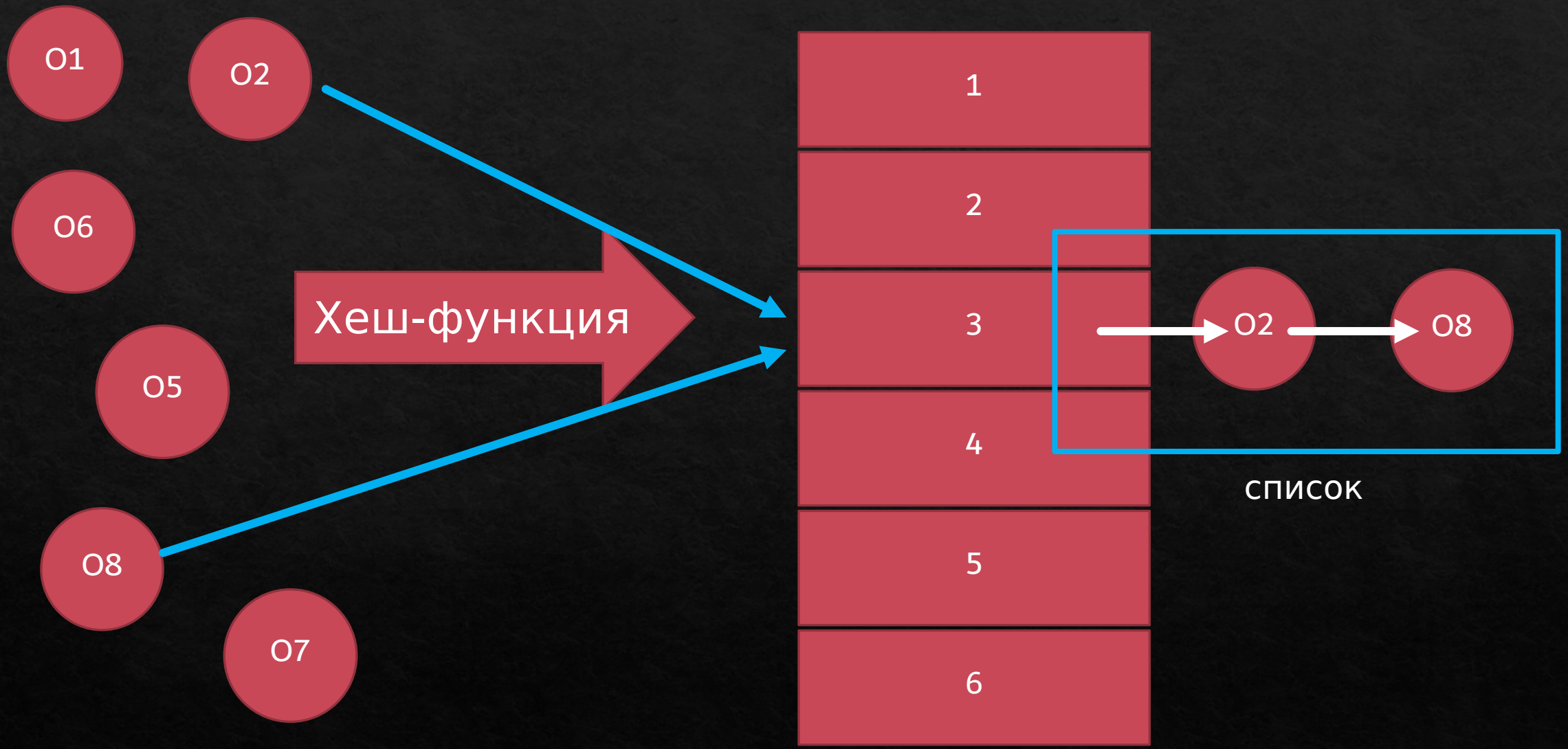
```
 3  struct Point {
 4      unsigned x, y;
 5  };
 6
 7  bool equal_point(Point const &lha, Point const &rha) {
 8      return lha.x == rha.x and lha.y == rha.y;
 9  }
10
11  unsigned hash_point(Point const &p) {
12      return (p.x << 16) | (p.y & 0xFFFF);
13  }
14
15  void print_point(Point const &p) {
16      std::cout << "(" << p.x << "," << p.y << ")";
17  }
```

Что такое коллизия в хеш-таблице?



Метод «цепочек»

Метод цепочек



Список и хеш-таблица

```
19  struct ListPair {
20      ListPair *tail;
21      Point p;
22  };
23
24  struct Table {
25      ListPair **table;
26      float load_factor;
27      std::size_t size;
28  };
```

```
43  Table new_table(std::size_t size) {
44      ListPair **table = new ListPair*[size];
45      for (std::size_t idx = 0; idx != size; ++idx)
46          table[idx] = nullptr;
47      return Table{table, 0.f, size};
48  }
49
50  void erase_table(Table &t) {
51      for (std::size_t idx = 0; idx != t.size; ++idx)
52          erase_list(t.table[idx]);
53      delete[] t.table;
54  }
```

Добавляем элемент

```
64  void add_element(Table &t, Point const &p) {
65      unsigned idx = hash_point(p) % t.size;
66      ListPair *head = t.table[idx];
67      bool found = false;
68      while (nullptr != head && !found) {
69          found = equal_point(head->p,p);
70          head = head->tail;
71      }
72      if (!found) {
73          t.table[idx] = add_before_list(t.table[idx], p);
74          t.load_factor += 1.f / t.size;
75      }
76  }
```

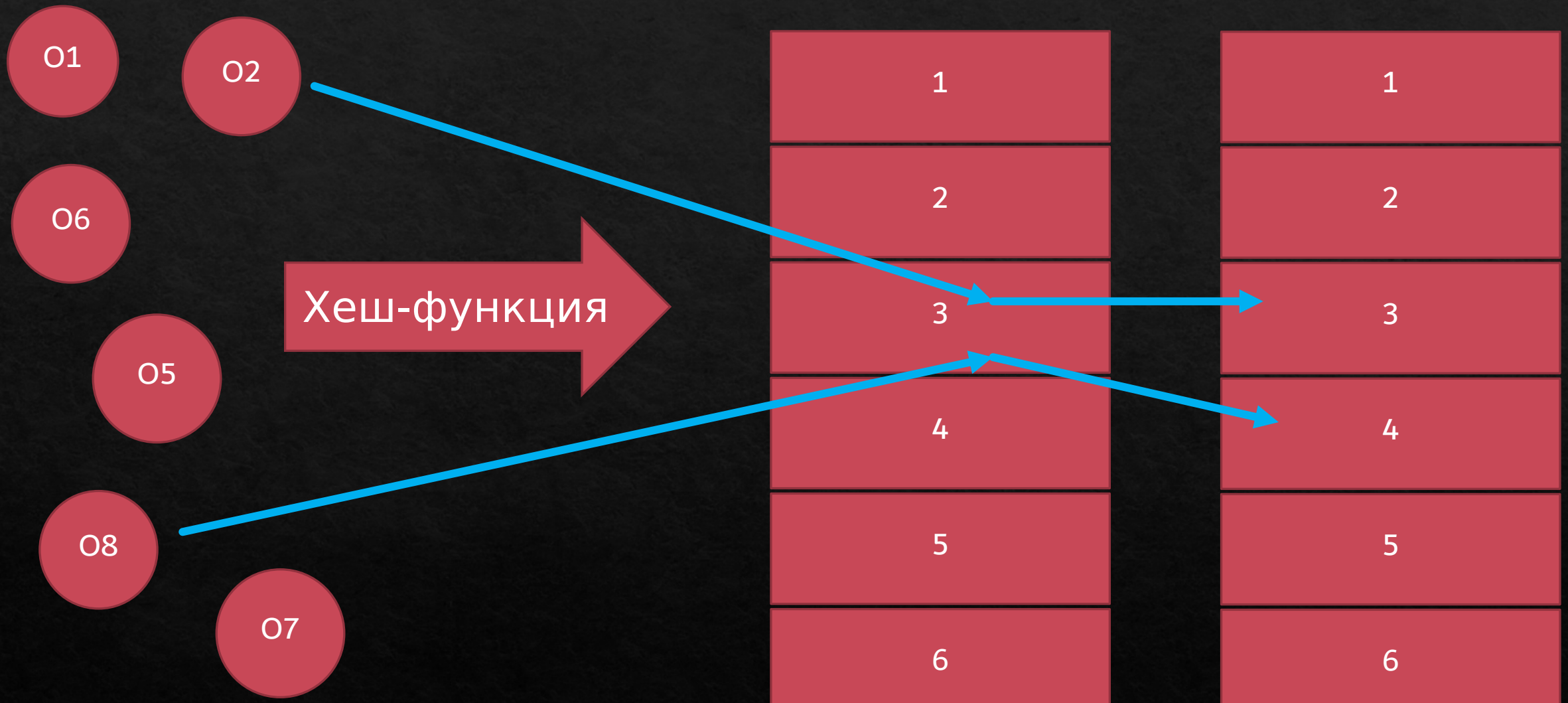
```
10 0.7
NULL
(1,15) ->
(3,4) ->
(3,15) ->
NULL
NULL
NULL
(4,3) -> (2,15) ->
(1,2) ->
(1,3) ->
```

Расширяем таблицу (rehash)

```
78  void expand_table(Table &t, std::size_t new_size) {
79      Table empty_table = new_table(new_size);
80      for (std::size_t idx = 0; idx != t.size; ++idx) {
81          ListPair *head = t.table[idx];
82          while (nullptr != head) {
83              add_element(empty_table, head->p);
84              head = head->tail;
85          }
86      }
87      erase_table(t);
88      t = empty_table;
89  }
```

```
23 0.304348
NULL
(1,15) ->
NULL
NULL
NULL
NULL
NULL
NULL
NULL
(3,4) ->
NULL
(2,15) ->
(1,2) ->
(1,3) ->
NULL
NULL
NULL
NULL
(4,3) ->
NULL
NULL
(3,15) ->
NULL
NULL
NULL
```

Метод «открытого адреса»


O1
O2
O6
Хеш-функция
O5
O8
O7
1
2
3
4
5
6
1
2
3
4
5
6

Хеш-таблица vs Дерево

\+

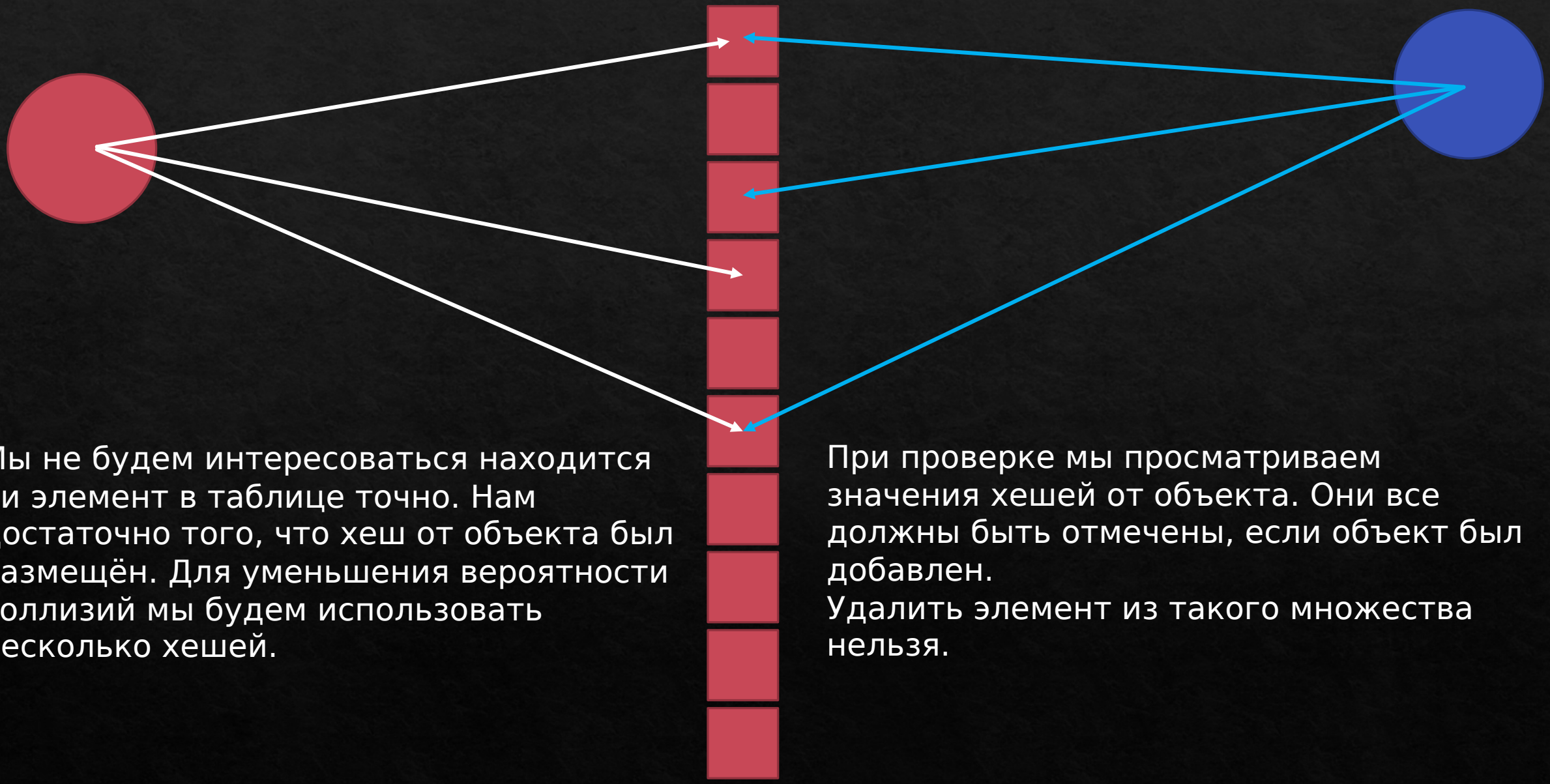
- Не требует определения порядка между объектами
- Операции выполняются с одинаковой амортизированной скоростью вне зависимости от количества элементов

\-

- Возможность коллизий сложно предсказать и сбалансировать
- Требует больше памяти при низких коэффициентах заполнения

Во всех ситуациях, когда нет требования на поиск диапазона упорядоченных элементов, следует предпочесть хеш-таблицу дереву.

Фильтр Блума



Мы не будем интересоваться находится ли элемент в таблице точно. Нам достаточно того, что хеш от объекта был размещён. Для уменьшения вероятности коллизий мы будем использовать несколько хешей.

При проверке мы просматриваем значения хешей от объекта. Они все должны быть отмечены, если объект был добавлен.
Удалить элемент из такого множества нельзя.